

Synthetic Difference-in-Differences Tutorial

Brady Earley

Date: May 28, 2026

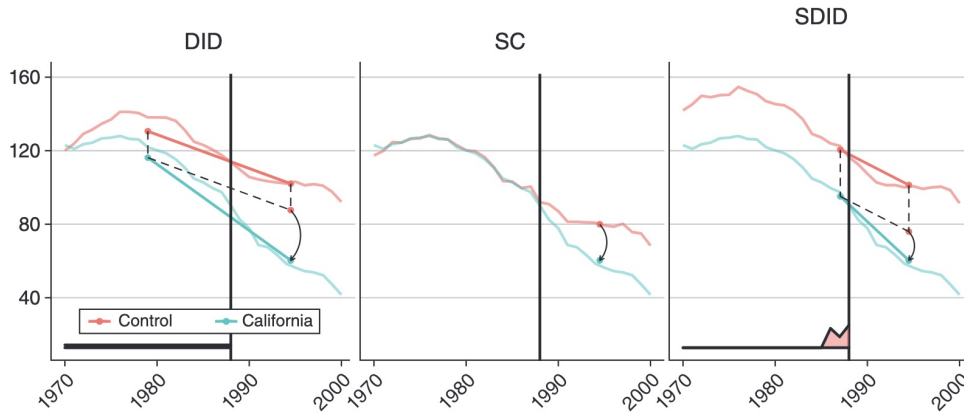
Problem Setup

Government officials, researchers, and the public care about creating law and policy that achieves its stated goals. But evaluating policy is challenging due to our inability to observe what would have happened if the policy had never been passed. One way to begin addressing this challenge is to use panel data where we can observe outcomes in units before and after the policy is enacted. By comparing this group to units where the policy was not enacted, we can estimate an average treatment effect from the difference between the change in treated unit and the change in the control unit. This is the Difference-in-Differences (DID) estimator (see Ashenfelter & Card, 1985; Card & Krueger, 1994). More formally, in a balanced panel with N units and T time periods with exposure to some binary treatment policy $W_{it} \in \{0, 1\}$, we write the DID estimator as the solution to a two-way fixed effects regression problem:

$$(\hat{\tau}^{did}, \hat{\mu}, \hat{\alpha}, \hat{\beta}) = \arg \min_{\alpha, \beta, \mu, \tau} \left\{ \sum_{i=1}^N \sum_{t=1}^T (Y_{it} - \mu - \alpha_i - \beta_t - W_{it}\tau)^2 \right\} \quad (1)$$

Where the outcome for unit i in period t is denoted with Y_{it} , a global intercept μ , a unit fixed effect α_i , a time fixed effect β_t and a treatment effect τ .

DID is particularly useful in many US policy settings because it is common to see subnational units like states or counties adopting different policies. DID relies on the parallel trends assumption - absent treatment, outcomes in the treated unit would have followed a parallel path to control units. In many empirical settings, the parallel trends assumption is messy and often challenging to validate for purposes of causal inference as shown below.¹



¹This figure is reproduced from Arkhangelsky, et al. 2021, Figure 1.

One popular way to address the parallel trends concern in DID is to create a synthetic control group (Abadie & Gardeazabal, 2003; Abadie, Diamond, & Hainmueller, 2010). The synthetic control is generated from a weighted average of predictors that conforms to a convex hull. The convex hull condition means that weights cannot be negative and must sum to 1. In compliance with these conditions, synthetic control weights ω are selected with the goal of minimizing predicted mean squared error in the pre-period based on pre-treatment characteristics X including outcomes.

$$\min_{\omega} (X_1 - X_0\omega)'V(X_1 - X_0\omega) \quad (2)$$

where X_1 is a vector of pre-treatment characteristics in the treated, X_0 is a matrix of the same pre-treatment characteristics in control, and V is a symmetric and positive semidefinite weighting matrix. This produces the following estimator of the treatment effect.

$$(\hat{\tau}^{sc}, \hat{\mu}, \hat{\beta}) = \arg \min_{\mu, \beta, \tau} \left\{ \sum_{i=1}^N \sum_{t=1}^T (Y_{it} - \mu - \beta_t - W_{it}\tau)^2 \hat{\omega}_i^{sc} \right\} \quad (3)$$

Thus, the synthetic control group increase confidence in the parallel trends assumption by creating a control group that closely approximates the treated group prior to the policy intervention.

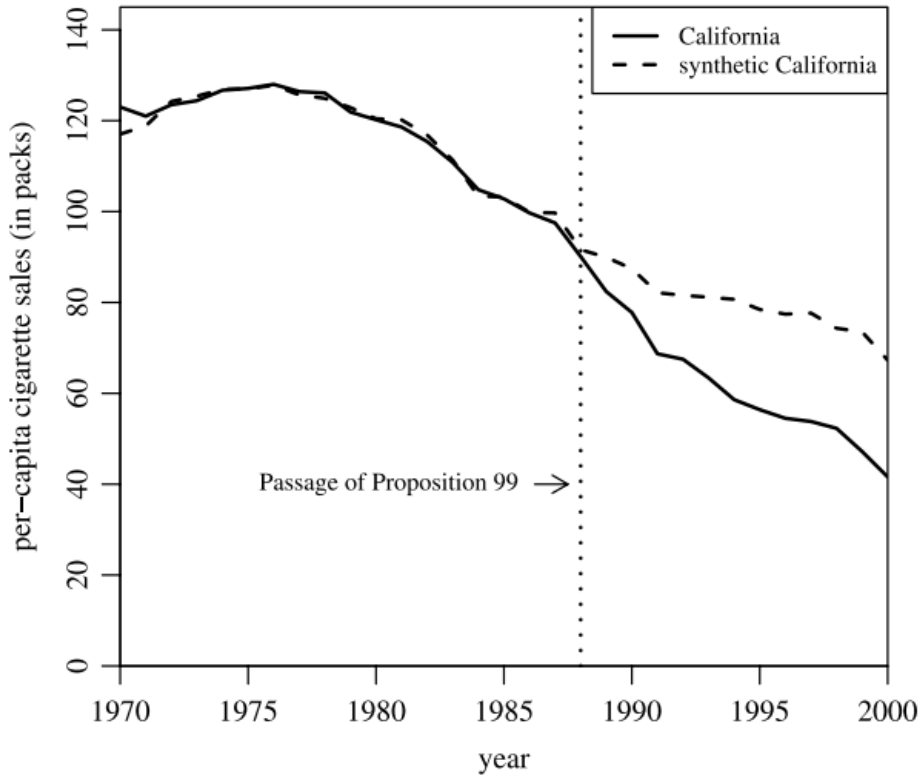


Figure 1: Figure 2 from Abadie, Diamond, Hainmueller (2010)

However, for the synthetic control method to work, a close approximation of outcomes in pre-treatment treated units from weighted combinations of the pre-treatment control units is needed. This means that weights for J control units are constrained by $\omega_j \geq 0$ and $\sum_{j=1}^J \omega_j = 1$.

In other words, if the pre-treatment outcomes in the treated group fall outside the convex hull of pre-treatment control weights, the synthetic control will perform poorly when estimating the counterfactual outcomes of the policy. And even when we get an excellent pre-treatment fit, there may be concerns about overfitting and thus failing to provide reliable estimates of counterfactual outcomes (see Abadie 2021).

Proposed Solution

In response to concerns about both DID and synthetic controls, Arkhangelsky, Athey, Hirshberg, Imbens, and Wager (2021) have proposed the synthetic difference-in-differences (SDID) approach - an alternative that combines the strengths of both methods while avoiding their pitfalls.

The estimand is the average treatment effect on the treated units (ATT) during the periods they were treated. More formally,

$$\tau = \frac{1}{N_{tr}T_{post}} \sum_{i=N_{co}+1}^N \sum_{t=T_{pre}+1}^T \tau_{it} \quad (4)$$

where individual treatment effects τ_{it} are averaged for treated units N_{tr} in the post-treatment period T_{post} .

As with DID and synthetic control, SDID relies on control group trends to estimate the counterfactual treated trends in the post-treatment period. Like DID, the SDID approach is “invariant to additive unit-level shifts” so it does not depend on matching pre-trend levels like the synthetic control. And like synthetic control, SDID finds weights that align pre-treatment characteristics in the treated with those in control. Thus, SDID can more credibly meet the parallel trends assumption than DID while identifying unit weights that match pre-treatment *trends* between groups (as opposed to matching levels in synthetic control).

Formally we assign unit weights by solving the following

$$\hat{\omega}_0, \hat{\omega}_{sdid} = \arg \min_{\omega_0 \in \mathbb{R}, \omega \in \Omega} l_{unit}(\omega_0, \omega) \quad (5)$$

where we minimize the loss function l_{unit} that measures fit between unit weights ω for the treatment units N_{tr} and control units N_{co} in the pre-treatment period T_{pre} .

$$l_{unit}(\omega_0, \omega) = \sum_{t=1}^{T_{pre}} (\omega_0 + \sum_{i=1}^{N_{co}} \omega_i Y_{it} - \frac{1}{N_{tr}} \sum_{i=N_{co}+1}^N Y_{it})^2 + \zeta^2 T_{pre} \|\omega\|_2^2$$

Notice that the loss function includes a scaled regularization parameter ζ to “match the size of a typical one-period outcome change Δ_{it} for unexposed units in the pre-period” (Arkhangelsky, et al. 2021). In full, the regularization parameter ζ is expressed as follows:

$$\zeta = (N_{tr}T_{post})^{\frac{1}{4}} \hat{\sigma} \quad (6)$$

where

$$\hat{\sigma}^2 = \frac{1}{N_{co}(T_{pre}-1)} \sum_{i=1}^{N_{co}} \sum_{t=1}^{T_{pre}-1} (\Delta_{it} - \bar{\Delta})^2$$

with

$$\Delta_{it} = Y_{i(t+1)} - Y_{it}$$

and

$$\bar{\Delta} = \frac{1}{N_{co}(T_{pre} - 1)} \sum_{i=1}^{N_{co}} \sum_{t=1}^{T_{pre}-1} \Delta_{it}$$

The scale used by the authors for regularization $(N_{tr}T_{post})^{\frac{1}{4}}$ is a “theoretically motivated” choice to balance the bias-variance tradeoff between DID (higher bias, lower variance) and SC (lower bias, higher variance). The regularization parameter ζ thus penalizes more extreme weights before aggregating to one, thereby reducing potential for bias while preserving more uniformity in the weights.

Beyond the unit weights, SDID also adds a time weight λ_t to improve pre-treatment fit between treated and control units. As before, we minimize a loss function to balance time weights between treated and control in the pre-period.

$$\hat{\lambda}_0, \hat{\lambda}_{sdid} = \arg \min_{\lambda_0 \in \mathbb{R}, \lambda \in \Lambda} l_{time}(\lambda_0, \lambda) \quad (7)$$

where

$$l_{time}(\lambda_0, \lambda) = \sum_{i=1}^{N_{co}} (\lambda_0 + \sum_{t=1}^{T_{pre}} \lambda_t Y_{it} - \frac{1}{T_{post}} \sum_{t=T_{pre}+1}^T Y_{it})^2$$

Bringing it all together, our SDID estimator can be written as follows:

$$(\hat{\tau}^{sdid}, \hat{\mu}, \hat{\alpha}, \hat{\beta}) = \arg \min_{\tau, \mu, \alpha, \beta} \left\{ \sum_{i=1}^N \sum_{t=1}^T (Y_{it} - \mu - \alpha_i - \beta_t - W_{it}\tau)^2 \hat{\omega}_i^{sdid} \hat{\lambda}_t^{sdid} \right\} \quad (8)$$

Implementation

This section demonstrates an implementation of the SDID estimator proposed by Arkhangelsky, et al. (2021). I implement the SDID estimator by hand using the algorithm from Arkhangelsky, et al. (2021, p. 4093):

1. Compute regularization parameter ζ .
2. Compute unit weights $\hat{\omega}^{sdid}$.
3. Compute time weights $\hat{\lambda}^{sdid}$.
4. Compute SDID estimator $\hat{\tau}^{sdid}$ using Equation (8).

I use the code provided in the `synthdid` package as a guide for my by-hand implementation. The documentation for the `synthdid` package is available [here](#).

Step 0: Data and Policy Setting

The Adoption and Foster Care Analysis Reporting System (AFCARS) foster care dataset is compiled every fiscal year by the Children’s Bureau within the Department of Health and Human Services. AFCARS collects case-level information about children entering and exiting foster care including age, race, and living arrangements. More information on the data documentation and access is available [here](#).

The panel of AFCARS data used for this tutorial covers reporting from all counties with more than 1000 foster child cases between 2000 to 2019. In 2006, Catholic Charities of Boston (CCB) failed to renew their foster care contract with Massachusetts in response to a nondiscrimination law prohibiting discrimination on the basis of sexual orientation. What was the effect of CCB’s removal on foster care adoptions?

CCB had been operating in five counties in Massachusetts prior to 2006: Suffolk, Essex, Middlesex, Bristol, Worcester. The dataset here includes a cleaned version of the AFCARS data that has removed counties with less than 1000 foster child cases and only includes the following case characteristics for a given year: averages for age(`fost_age`), sex(`female`), and race (`blkafram`) of the county's foster children, median subsidy amount provided to adoptive parents the county (`county_sub`), and the proportion of foster children with disabilities (`clindis`).

Number of counties: 37

Number of years: 20

Average County Foster Care Adoption Rate (Normalized by Adoptive Pool): 30.45 %

The table below displays six years of county data from Baltimore, Maryland (24150) and six years from Bristol, Massachusetts (25005).

fipscode	repdatyr	norm_adopt	treat	post	fost_age	county_sub	female	clindis	blkafram
24510	2003	0.1958713	0	0	8.032165	535.0	0.4743133	0.2039949	0.9025370
24510	2004	0.2271498	0	0	8.059491	535.0	0.4872010	0.2189554	0.8942273
24510	2005	0.1447188	0	0	8.188615	535.0	0.4851435	0.2232899	0.8932861
24510	2006	0.2203245	0	1	6.876174	0.0	0.4903025	0.5907268	0.8974776
24510	2007	0.2208259	0	1	6.803411	0.0	0.4903035	0.4763473	0.8971285
24510	2008	0.2983114	0	1	6.668856	196.0	0.4872243	0.4435301	0.8989819
25005	2003	0.2191142	1	0	5.342658	462.5	0.5129698	0.0957166	0.1697515
25005	2004	0.1808036	1	0	5.376392	520.0	0.5204461	0.0966543	0.1741901
25005	2005	0.2665474	1	0	5.633274	546.0	0.5013812	0.1606075	0.1836171
25005	2006	0.2341137	1	1	5.637124	546.0	0.4780784	0.1445221	0.1897436
25005	2007	0.2495697	1	1	5.526678	546.0	0.4784712	0.1707789	0.1857765
25005	2008	0.2266187	1	1	5.877698	557.0	0.4773969	0.2213439	0.2014925

The dependent variable `norm_adopt` is a normalized measure of foster children who were adopted in a particular year out of the total number of foster children with a case goal of adoption. For many foster children, the primary is establishing permanency. When possible, children will be reunified with their family. However, when this option is no longer in the best interests of the child, adoption is the next best step for achieving permanency.

The following code prepares the data for estimation by defining the county unit labels and the outcome matrix.

```
library(tidyr)
library(haven)

# vector of county names
counties <- c("Jefferson, AL", "Pima, AZ", "Contra Costa, CA", "Los Angeles, CA", "Riverside, CA", "Sacramento, CA")

# reshape data to conform to SDID estimation format
Y_df <- pivot_wider(df, id_cols = fipscode, names_from = repdatyr, values_from = norm_adopt)

# clean rownames
Y_df$fipscode <- as.character(zap_labels(Y_df$fipscode))

# remove ID column and convert into matrix
```

```

Y <- as.matrix(Y_df[,-1])

# set rownames
rownames(Y) <- counties

# define rows and columns
N = nrow(Y)
T = ncol(Y)

```

Step 1: Regularization

This code calculates the regularization parameter by hand, as outlined in Arkhangelsky, et al. (2021). Note that the `synthdid` package function for SDID estimation uses a slightly different approach. Instead of using variance demeaned within each unit (sigma hat squared), the package uses the standard deviation pooled across all units. I use the by-hand approach here to illustrate the difference, but each end up with very similar estimates.

```

library(dplyr)

# compute first sum result for delta bar
sum_result <- df |>
  arrange(fipscode, repdatyr) |>
  group_by(fipscode) |>
  mutate(delta = lead(norm_adopt) - norm_adopt) |>
  filter(post == 0, lead(post) == 0,
         treat == 0, lead(treat) == 0) |>
  summarise(sum_i = sum(delta, na.rm = TRUE), .groups = "drop") |>
  summarise(sum_it = sum(sum_i, na.rm = TRUE))

# create objects for other variables in equation
T_pre <- length(unique(df$repdatyr[df$post==0]))
N_co <- length(unique(df$fipscode[df$treat==0]))
T_post <- length(unique(df$repdatyr[df$post==1]))
N_tr <- length(unique(df$fipscode[df$treat==1]))

# calculate delta bar and delta
delta_bar <- (1/(N_co*(T_pre - 1)))*sum_result
delta <- lead(df$norm_adopt) - df$norm_adopt

# calculate sums for sigma_hat
new_sum_result <- df |>
  arrange(fipscode, repdatyr) |>
  group_by(fipscode) |>
  mutate(delta_diff = (delta - delta_bar)^2) |>
  filter(post == 0, lead(post) == 0,
         treat == 0, lead(treat) == 0) |>
  summarise(sum_i = sum(delta_diff, na.rm = TRUE), .groups = "drop") |>
  summarise(sum_it = sum(sum_i, na.rm = TRUE))

sigma_hatsq <- (1/(N_co*(T_pre - 1)))*new_sum_result

# put it all together
zeta <- as.numeric(((N_tr*T_post)^(0.25))*sqrt(sigma_hatsq))

```

Step 2: Unit Weights

Using the regularization parameter from above, we can generate and optimize unit weights. This first code chunk defines the necessary parameters for solving the unit weight optimization problem.

```
# initial empty list for unit and time weights
weights = list(omega = NULL, lambda = NULL)

# intercept for unit weights
omega_intercept = TRUE

# function for dimension reduction used in solving optimization problem for each of the weights
sparsify_function = function(v) { v[v <= max(v)/4] = 0; v/sum(v) }
sparsify = sparsify_function

# defining max iterations and minimum decrease for sparsify function (using noise level as defined by t
max_iter = 1e4
max_iter_pre_sparsify = 100
noise_level = sd(apply(Y[1:N_co,1:T_pre], 1, diff))
min_decrease = 1e-5 * noise_level
```

Both unit and time weight optimization uses the Frank-Wolfe algorithm for constrained convex optimization, making it a good fit for the convex hull constraint in synthetic control and SDID estimation. This algorithm is built into the `synthdid` package and is directly imported from that package below.

```
# create a function to generate an outcome matrix (Y) defined by averaging treated units into a row and
collapsed.form <- function(Y, NO, TO) {
  N = nrow(Y)
  T = ncol(Y)

  # Average treated units (rows NO+1 to N) into one row
  treated_avg = colMeans(Y[(NO+1):N, , drop=FALSE])

  # Combine control rows with the averaged treated row
  Y_collapsed = rbind(Y[1:NO, ], treated_avg)

  # Average post-treatment columns (TO+1 to T) into one column
  post_avg = rowMeans(Y_collapsed[, (TO+1):T, drop=FALSE])

  # Combine pre-treatment columns with the averaged post column
  Y_out = cbind(Y_collapsed[, 1:TO], post_avg)

  return(Y_out)
}

Yc = collapsed.form(Y, N_co, T_pre)

# generate unit weights with Frank-Wolfe optimization algorithm and sparsify function from above.
omega_opt = synthdid:::sc.weight.fw(t(Yc[, 1:T_pre]),
                                   zeta = zeta,
                                   intercept = omega_intercept,
                                   lambda = weights$omega,
                                   min.decrease = min_decrease,
                                   max.iter = max_iter_pre_sparsify)
```

```

# second pass to re-optimize
omega_opt = synthdid::sc.weight.fw(t(Yc[, 1:T_pre]),
                                   zeta = zeta,
                                   intercept = omega_intercept,
                                   lambda = sparsify(omega_opt$lambda),
                                   min.decrease = min_decrease,
                                   max.iter = max_iter_pre_sparsify)

weights$omega = omega_opt$lambda
weights$omega_vals = omega_opt$vals

```

Note the transpose is used in the optimization function to make time periods rows and units as columns.

Step 3: Time Weights

Time weights follow the same structure as unit weights except without the regularization parameter as shown below.

```

# nuisance parameters for time weights
eta_lambda = 1e-6
zeta_lambda = eta_lambda * noise_level

# time weights intercept placeholder
lambda_intercept = TRUE

# following the same structure as the unit weights but using time weight parameters
lambda_opt = synthdid::sc.weight.fw(Yc[1:N_co, ],
                                   zeta = zeta_lambda,
                                   intercept = lambda_intercept,
                                   lambda = weights$lambda,
                                   min.decrease = min_decrease,
                                   max.iter = max_iter_pre_sparsify)

# second pass to re-optimize
lambda_opt = synthdid::sc.weight.fw(Yc[1:N_co, ],
                                   zeta = zeta_lambda,
                                   intercept = lambda_intercept,
                                   lambda = sparsify(lambda_opt$lambda),
                                   min.decrease = min_decrease,
                                   max.iter = max_iter_pre_sparsify)

weights$lambda = lambda_opt$lambda
weights$vals = lambda_opt$vals

```

Step 4: SDID Estimator

With time and unit weights, we are ready to solve our SDID estimator. The estimating equation multiplies outcomes by a unit-weighted (left part) and time-weighted (right part) average. Note

```

estimate = t(c(-weights$omega, rep(1 / N_tr, N_tr))) %*% Y %*% c(-weights$lambda, rep(1 / T_post, T_post))
estimate

```

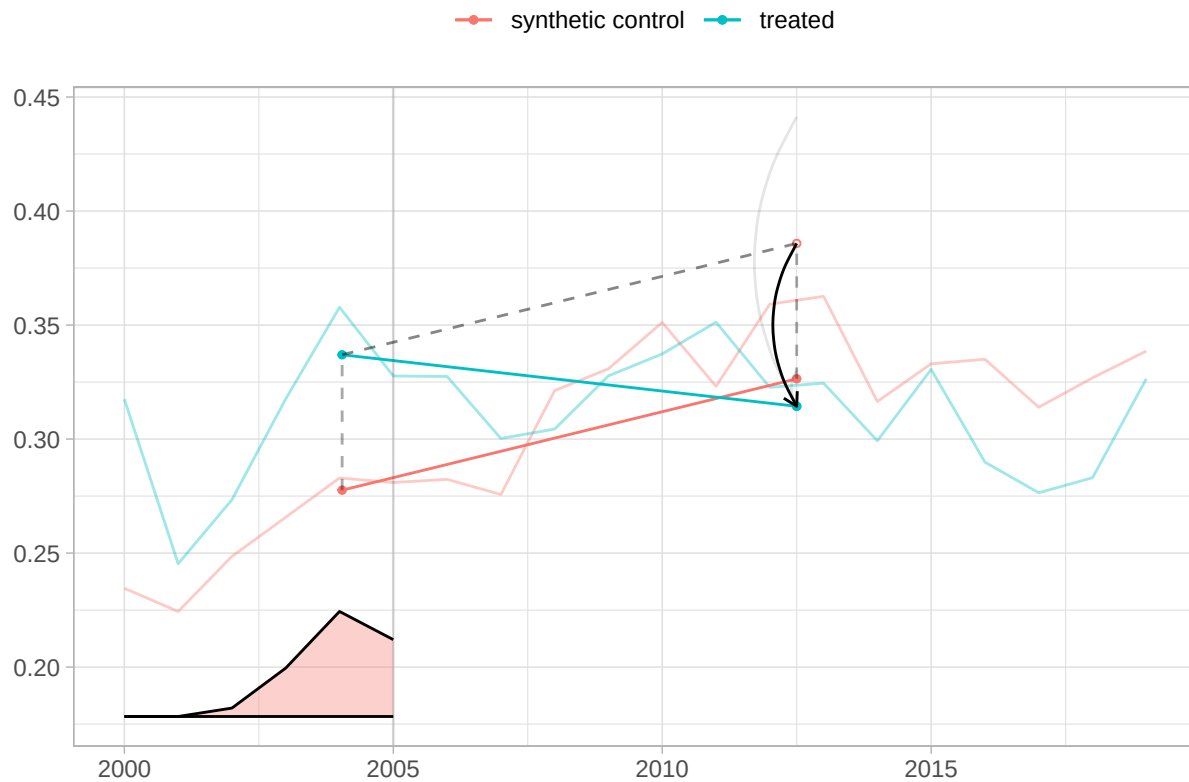
```
##           [,1]
## [1,] -0.06533834
```

We can do the same thing with the out-of-the box `synthdid` package. Notice that the canned function is slightly different from the by-hand calculation due to the differences in calculating ζ .

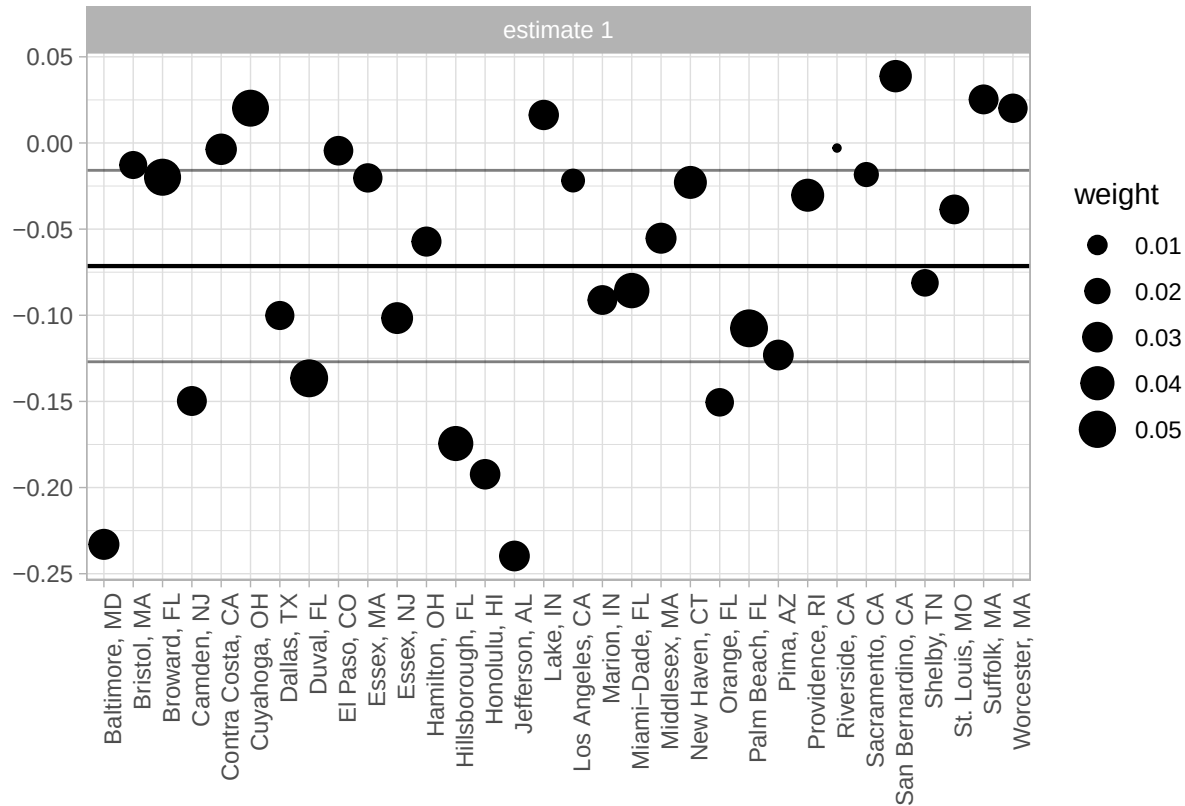
```
library(synthdid)
# estimate SDID and plot with weights
tau_hat = synthdid_estimate(Y, N_co, T_pre)
tau_hat
```

```
## synthdid: -0.071 +- 0.056. Effective N0/N0 = 28.8/32~0.9. Effective T0/T0 = 2.9/6~0.5. N1,T1 = 5,14.
```

```
plot(tau_hat)
```



```
synthdid_units_plot(tau_hat)
```



Arkhangelsky, et al (2021) also mentions that we could include our covariate set in the SDID estimation by regressing the outcome variable onto the covariates and then using the residualized outcome as our Y in the SDID estimation. This is an optional extension that may improve pre-treatment fit. Note that this code chunk may take a couple minutes to run.

```
# create covariate matrix
K = 4
X = array(dim = c(N, T, K))
X[, , 1] = df$fost_age
X[, , 2] = df$white
X[, , 3] = df$county_sub
X[, , 4] = df$clindis

# put county sub on a comparable scale to other covariates by standardizing
for(k in 1:K) {
  X[, , k] = (X[, , k] - mean(X[, , k])) / sd(X[, , k])
}

# re-estimate with covariates
tau_hat2 = synthdid_estimate(Y, N_co, T_pre, X = X)
tau_hat2
```

```
## synthdid: -0.084 +- 0.058. Effective N0/N0 = 30.4/32~0.9. Effective T0/T0 = 3.2/6~0.5. N1,T1 = 5,14.
```

Conclusion

The SDID estimator is a helpful tool for combining features of DID and SC approaches to policy analysis. It is particularly valuable when interested in the ATT for a particular time period - a common focus for policy

and program evaluation. The availability of state and county-level data make this a feasible approach in many data settings as well. In the foster care context presented above, data were publicly available and made relatively accessible to any researcher. Paired with a relatively straightforward package for implementation, SDID is primed to assist researchers with making new, important, and valid causal claims.

AI Assistance

I used Chat GPT to debug R Markdown knitting errors and Claude to debug R code errors and warnings. Claude also provided prototype code for the `collapsed.form` helper function in Step 2 and the covariate standardization in Step 4. Claude also helped with explanation of portions of the original code in the `synthdid` package.

References

- Abadie, A. 2021. Using Synthetic Controls: Feasibility, Data Requirements, and Methodological Aspects. *Journal of Economic Literature* 59(2): 391-425.
- Abadie, A., A. Diamond, and J. Hainmueller. 2010. Synthetic Control Methods for Comparative Case Studies: Estimating the Effect of California’s Tobacco Control Program. *Journal of the American Statistical Association* 105(490): 493–505.
- Abadie, A., and J. Gardeazabal. 2003. The Economic Costs of Conflict: A Case Study of the Basque Country. *American Economic Review* 93(1): 113–132
- Arkhangelsky, D., S. Athey, D. A. Hirshberg, G. W. Imbens, and S. Wager. 2021. Synthetic Difference-in-Differences. *American Economic Review* 111(12): 4088–4118. <https://www.aeaweb.org/articles?id=10.1257/aer.20190159>.
- Ashenfelter, O. and D. Card. Using the Longitudinal Structure of Earnings to Estimate the Effect of Training Programs. 1985. *Review of Economics and Statistics* 67(4): 648-660.
- Card, D. and A.B. Krueger. Minimum Wages and Employment: A Case Study of the Fast-Food Industry in New Jersey and Pennsylvania. 1994. *American Economic Review* 84(4): 772-793.
- Clarke, D., D. Palaiñir, S. Athey, and G. Imbens. Synthetic Difference in Differences Estimation. 2023. <https://arxiv.org/pdf/2301.11859>.